Part 3：OpenClaw 安全防护入门

> *你的 AI 助手能力越强，一旦被利用，破坏力也越大。这一章用 15 分钟，给你的 OpenClaw 穿上"防弹衣"。*

引言

2026 年 2 月，安全研究机构 Oasis Security 披露了一个名为 **ClawJacked** 的严重漏洞——任何网站都可以通过 WebSocket 暴力破解密码，连接到你电脑上运行的 OpenClaw Gateway，然后**完全控制你的 AI 助手**：读写文件、执行命令、窃取数据。OpenClaw 团队在 24 小时内紧急修复，但全球数万用户已经暴露在攻击窗口中。

同月，另一个名为 **ClawHavoc** 的恶意事件被 Koi Security 曝光——ClawHub 技能市场上竟然藏着 **341 个恶意 Skill**，伪装成正常的加密货币工具、YouTube 插件，实际在背后窃取你的 API Key、植入后门、甚至下载恶意软件到你的电脑里。

这不是科幻小说，这是两个月前真实发生的安全事件。

上一章我们让 OpenClaw 开口说了第一句话，还让它帮我们执行了系统命令。但正因为它能操作你的电脑、读写你的文件、调用你的 API——**能力越大，风险越大**。如果不做好安全防护，你的 AI 助手随时可能变成别人的"内鬼"。

这一章，我们从六个维度给你的 OpenClaw 建立安全防线：安装安全、权限控制、Skill 安全、网络安全、行为控制、成本控制。学完之后，你会拿到一份**安全基线配置**——相当于 OpenClaw 的"防弹衣"。

3.1 安装安全：仅从官方渠道安装

官方安装渠道

OpenClaw 只有两个值得信任的安装来源：

渠道	安装方式	安全保障
npm 官方包	`npm install -g openclaw`	npm 审计机制 + SHA-512 校验
GitHub Release	从 openclaw/openclaw 仓库下载	源码可审计 + Release 签名

就这两个，**没有第三个**。任何号称"更方便的安装工具"、"一键脚本"、"安装包下载"——在没有验证来源之前，都应该保持警惕。

真实案例警示：ClawHavoc 恶意安装工具

为什么要如此强调安装渠道？因为有人已经利用这个信任盲区发起了真实攻击。

2026 年 2 月，安全公司 Koi Security 在审计 ClawHub（OpenClaw 的技能市场）时发现：**335 个恶意 Skill 伪装成安装工具和前置依赖**，诱导用户下载一个名为 **Atomic Stealer（AMOS）** 的恶意软件。

攻击手法非常狡猾：

1. **制作伪装 Skill**——名字起得像正规工具，比如"OpenClaw Agent Installer"、"Quick Setup Helper"
2. **在 Skill 描述中声称需要先安装"前置依赖"**——给出一个下载链接
3. **链接指向伪造的安装包**——文件名叫 `openclaw-agent.zip`，或者是 glot.io 上的"安装脚本"
4. **用户运行后，AMOS 恶意软件悄悄安装**——它会窃取浏览器密码、加密货币钱包、Keychain 数据

> *AMOS（Atomic Stealer）是一款专门针对 macOS 的信息窃取恶意软件，2023 年首次出现，至今仍在活跃。它能读取 macOS Keychain 中存储的所有密码。*

安装验证清单

在你安装 OpenClaw（或任何工具）时，养成以下习惯：

- **验证版本号**：`openclaw --version` 的输出应与 npm/GitHub 上的最新版一致

- 运行安全审计：`npm audit` 检查已知漏洞
- 不下载可疑安装包：任何 `.zip` 或非官方渠道的安装脚本，一律不碰
- 保持更新：`npm update -g openclaw`（ClawJacked 漏洞就是在更新中修复的）

一句话总结：**不确定来源，就不要安装。**

3.2 权限控制：最小权限原则

权限控制是 OpenClaw 安全体系的骨架。上一章我们允许 AI 执行系统命令来查看硬件信息——但如果它想执行 `rm -rf ~` 呢？

这一节，我们深入了解 OpenClaw 的权限控制体系，并通过一个小实验来"感受"权限配置的实际效果。

工具权限体系

OpenClaw 的所有能力都通过"工具（Tools）"来提供——执行命令是一个工具、读文件是一个工具、浏览网页也是一个工具。**权限控制就是决定哪些工具可用、哪些被禁止。**

```
{
  "tools": {
    "profile": "coding",          // 预设配置
    "allow": ["read", "write"],   // 白名单：允许的工具
    "deny": ["exec", "browser"]   // 黑名单：禁止的工具（优先级高于 allow）
  }
}
```

OpenClaw 提供了三个预设 Profile，适合不同场景：

PROFILE	允许的工具	适用场景
`coding`	exec, process, read, write, edit, image, sessions 等	编程开发（默认）
`messaging`	sessions, channels 等（无 exec/write）	纯聊天/消息转发
`minimal`	极少工具	最安全，功能也最少

规则很简单：**deny 的优先级永远高于 allow**。 也就是说，如果一个工具同时出现在 allow 和 deny 列表中，它会被禁止。这是白名单和黑名单的双保险机制。

Shell 命令执行控制（tools.exec）

在所有工具中，`exec`（命令执行）是最危险也最有用的——它让 AI 可以在你的电脑上运行任何命令。OpenClaw 对它做了三层控制：

第一层：**security 模式**

模式	说明	效果
`deny`	完全禁止执行命令	AI 无法运行任何命令
`allowlist`	仅允许白名单中的命令	只有预先批准的命令才能运行
`full`	允许所有命令	AI 可以运行任何命令

第二层：**ask 确认模式**

模式	说明	效果
`off`	永不提示	AI 静默执行所有命令（危险！）
`on-miss`	不在白名单时提示	已知安全的命令直接执行，新命令弹确认
`always`	每次都提示	任何命令执行前都要你点"同意"

第三层：**safeBins（安全程序白名单）**

```
"safeBins": ["jq", "cut", "uniq", "head", "tail", "tr", "wc"]
```

这些是 OpenClaw 认为"只能通过 stdin 操作、不会直接访问文件系统"的安全程序。当 security 设为 `allowlist` 时，这些程序不需要额外审批。

用一个类比总结三层控制：

> ***security** 是门卫（决定谁能进大门）、**ask** 是前台（进来之后要不要登记）、**safeBins** 是 VIP 名单（名单上的人直接放行）。*

文件系统访问控制

除了命令执行，文件系统访问也需要控制。OpenClaw 通过工具白名单 + 沙箱组合实现：

```
{
  "tools": {
    "allow": ["read"],                          // 只允许读操作
    "deny": ["write", "edit", "apply_patch"]    // 禁止写操作
  },
  "agents": {
    "defaults": {
      "sandbox": {
        "workspaceAccess": "ro"     // "none" | "ro" | "rw"
      }
    }
  }
}
```

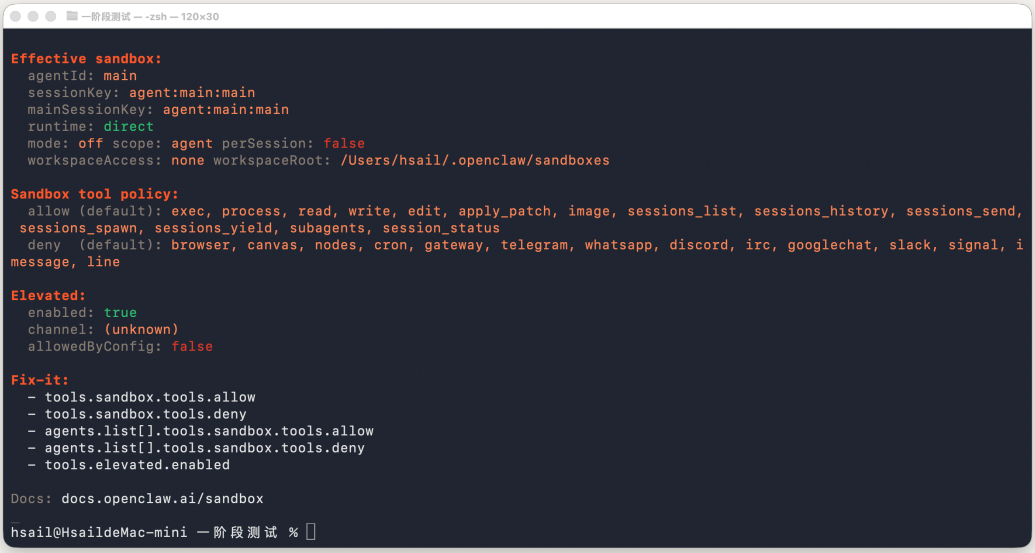
Workspace Access 三个级别：

模式	效果	适用场景
`none`	完全隔离，工具无法访问文件系统	纯对话 Agent
`ro`	只读挂载工作区	分析型 Agent
`rw`	读写挂载工作区	开发型 Agent

实操：查看当前有效策略

让我们看看你的 OpenClaw 当前实际生效的权限配置。在终端中运行：

```
$ openclaw sandbox explain
```



来解读几个关键信息：

项目	当前值	含义
runtime	`direct`	直接在宿主机运行（未启用 Docker 沙箱）
mode	`off`	沙箱关闭
allow	exec, process, read, write...	允许执行命令、读写文件等
deny	browser, canvas, cron...	禁止浏览器、画布、定时任务等
elevated	enabled, allowedByConfig: false	提权功能已启用但被 gate 拦截

这说明什么？ 当前配置是"个人开发环境"的典型状态——沙箱关闭、命令执行开放。对于你自己使用来说没问题，但如果要给别人使用或暴露到网络上，就需要加强了。

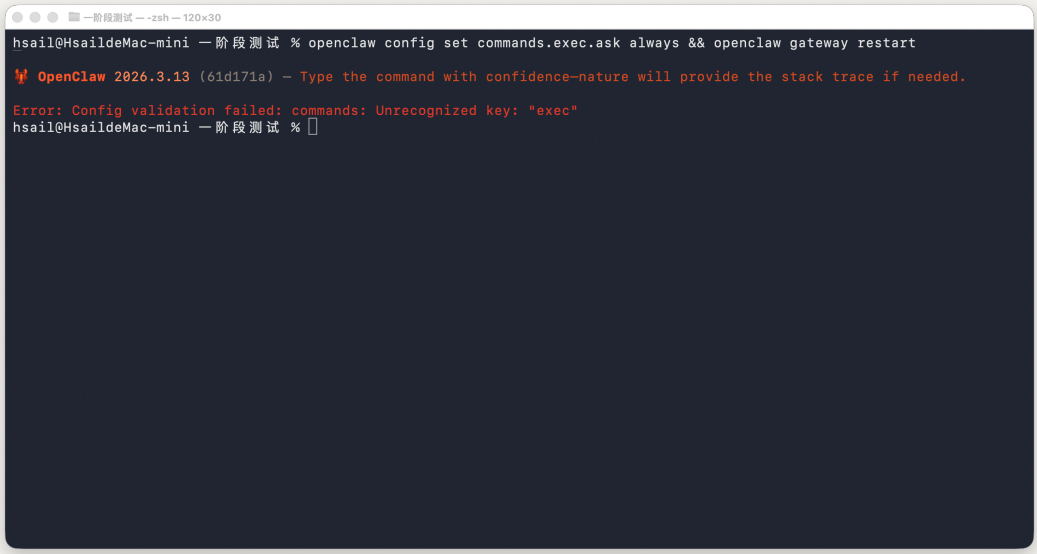
小实验：感受 exec.ask 的差异

光看配置表格不够直观。我们来做一个小实验，让你亲身感受 `ask` 设置对使用体验的影响。

第一步：把 **exec.ask** 设为 **`always`**

```
$ openclaw config set tools.exec.ask "always"
Updated tools.exec.ask. Restart the gateway to apply.

$ openclaw gateway restart
Restarted LaunchAgent: gui/501/ai.openclaw.gateway
```



第二步：在 **Dashboard** 中对话

打开 Dashboard，在对话框中输入：

```
帮我查看当前目录有哪些文件
```

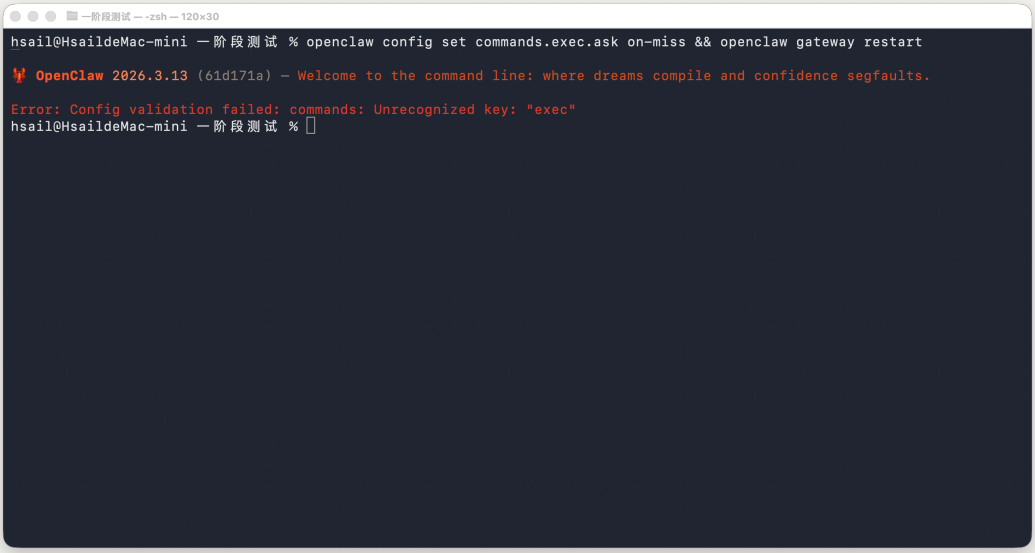
这一次，你会发现 AI 想执行 `ls` 命令时，Dashboard 弹出了一个审批对话框——即使 `ls` 是一个完全无害的命令，在 `always` 模式下也必须经过你的同意。

如果你继续对话让它做更多操作，每一个命令都会弹一次审批。体验就像每做一步都要签字确认——安全是安全了，但效率大打折扣。

第三步：恢复为 **`on-miss`** 模式

```
$ openclaw config set tools.exec.ask "on-miss"
Updated tools.exec.ask. Restart the gateway to apply.

$ openclaw gateway restart
Restarted LaunchAgent: gui/501/ai.openclaw.gateway
```



现在再试同样的对话。你会发现：`ls`、`cat` 这些常见命令**直接执行了**，不再弹确认；但如果 AI 想执行一个从未见过的新命令（比如 `curl` 或 `pip install`），它才会弹出审批框。

这就是 **`on-miss`** 的精髓——已知安全的命令放行，陌生命令才拦截。既保障安全，又不影响效率。

> *推荐配置：个人开发环境使用 `on-miss`（平衡安全与效率），多人共享环境使用 `always`（最大化安全）。绝对不要在个人电脑上设为 `off`——那等于裸奔。*

3.3 Skill 安全：谨慎安装第三方 Skill

如果说权限控制是"锁好自己家的门"，那 Skill 安全就是"别往家里带陌生人"。

OpenClaw 的 Skill 生态非常丰富——ClawHub 上有超过 13,000 个社区 Skill，覆盖编程、自动化、消息处理等各种场景。但繁荣的生态也意味着鱼龙混杂。

ClawHavoc：触目惊心的数字

让我们回到 ClawHavoc 事件，这次看更详细的数据。

Koi Security 对 ClawHub 进行了大规模安全审计，覆盖 **2,857 个 Skill**。结果让人倒吸一口凉气：

统计项	数据
审计 Skill 总数	2,857
恶意 Skill	**341**（占 11.9%）
存在安全缺陷（扩展审计）	3,401 / 9,234（占 36.82%）

也就是说，你每安装 10 个第三方 Skill，统计上就有 **1 个是恶意的**。

攻击类型分布

这 341 个恶意 Skill 使用了多种攻击手法：

攻击类型	数量	手法
AMOS 恶意软件	335	伪装为前置依赖，诱导下载 Atomic Stealer
凭证窃取	若干	通过 OAuth 代理拦截 API Token
反向 Shell	若干	在 Skill 中隐藏后门，建立远程连接
权限提升	若干	利用 Elevated 执行恶意命令

它们伪装成什么？看起来都很"正经"：

- 加密货币工具（"Crypto Portfolio Tracker"）
- YouTube 工具（"YT Auto Downloader"）
- Google Workspace 集成（"Gmail AI Assistant"）
- 自动更新器（"OpenClaw Auto Updater"）
- 金融工具（"Stock Alert Bot"）

> *安全机构 **Snyk** 的独立审计也印证了这一点：在他们审计的 Skill 样本中，**13.4%** 存在关键安全问题。*

平台审核不是万能的

ClawHub 已经采取了多项审核措施：

审核措施	效果
VirusTotal 扫描	2026.3 起集成，自动扫描上传的 Skill
AI 审查	内部 AI 模型审查 Skill 内容
静态分析引擎	2026.3.8 起开源
社区举报	3 人以上举报自动隐藏
发布门槛	GitHub 账号需注册满 1 周

但正如安全公司 CertiK 警告的：**平台扫描机制并非万无一失，深度隐藏的提示注入可能绕过检测。** 不能把安全全部托付给平台审核。

Skill 白名单配置

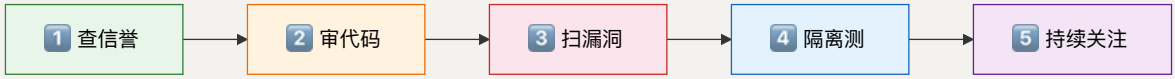
OpenClaw 提供了 Skill 级别的权限控制：

```
{
  "skills": {
    "allowBundled": ["gemini", "peekaboo"],    // 仅允许指定的内置 Skill
    "entries": {
      "image-lab": { "enabled": true },
      "sag": { "enabled": false }              // 禁用特定 Skill
    }
  },
  "tools": {
    "exec": {
      "autoAllowSkills": false    // ← 关键！不自动信任 Skill 引用的程序
    }
  }
}
```

特别注意 `autoAllowSkills` 这个配置——如果设为 `true`（默认），Skill 引用的任何可执行文件都会被自动加入白名单。这意味着一个恶意 Skill 可以让任何程序绕过审批直接运行。**建议设为 `false`。**

五步安全检查清单

在安装任何第三方 Skill 之前，建议走一遍这个清单：



步骤	操作	关注点
查信誉	检查作者的 GitHub 主页、Skill 下载量	下载量建议 1,000+，星数 50+
审代码	`openclaw skills info <name>` 查看 Skill 详情	关注 `prerequisites` 中的可疑下载链接
扫漏洞	使用 ClawSecure 等工具扫描	检查是否包含已知恶意代码特征
隔离测试	先在沙箱模式下运行	不让新 Skill 直接接触你的文件和命令
持续关注	定期检查安全公告	已安装的 Skill 也可能被发现问题

一句话原则：像对待手机 **App** 一样对待 **Skill——不是什么都往上装。**

3.4 网络安全：防止公网暴露

Gateway 是 OpenClaw 的大门。门关得好不好，决定了你的 AI 助手是只对你开放，还是对全世界开放。

Gateway 绑定模式

Gateway 有三种绑定模式，决定了"谁能连上来"：

模式	监听地址	风险等级	适用场景
`loopback`	127.0.0.1（仅本机）	低	个人本地使用（推荐）
`lan`	0.0.0.0（局域网所有接口）	中	局域网多设备访问
`tailnet`	Tailscale 虚拟网络	中	远程安全访问

为什么 `loopback` 是最安全的？

`127.0.0.1` 是一个特殊的"回环地址"——数据包永远不会离开你的电脑。即使黑客扫描了你的 IP 地址，也找不到 OpenClaw Gateway，因为它根本不对外开放。

公网暴露有多危险?

SecurityScorecard 的一份报告发现：全球有超过 **135,000 个 OpenClaw** 实例暴露在公网上，其中 **15,000+** 个存在远程代码执行漏洞。这意味着攻击者可以通过互联网直接连上这些 Gateway，控制背后的 AI 助手——读文件、执行命令、窃取数据。

> *这就好比你家大门不但没锁，还挂了个牌子写着"欢迎进来随便拿"。*

Gateway 认证

即使是 loopback 模式，也需要认证。OpenClaw 支持两种认证方式：

认证方式	说明	适用场景
`token`（默认）	自动生成长随机 Token	推荐
`password`	通过环境变量设置密码	需要自定义密码时

Token 模式的好处是**自动生成、足够长（不容易暴力破解）、通过 URL 参数传递（无需手动输入）**。

还记得上一章获取 Dashboard URL 时看到的 `#token=a261b163...`？那就是 Gateway 的认证 Token。

> *注意：Gateway 认证是**强制的**。如果你手动删除了 token 配置，Gateway 会拒绝所有 WebSocket 连接。这是 ClawJacked 漏洞修复后加强的安全措施。*

mDNS 发现服务

OpenClaw Gateway 默认通过 mDNS（多播 DNS，也叫 Bonjour）广播自身存在。这意味着同一局域网的其他设备可以发现你的 OpenClaw：

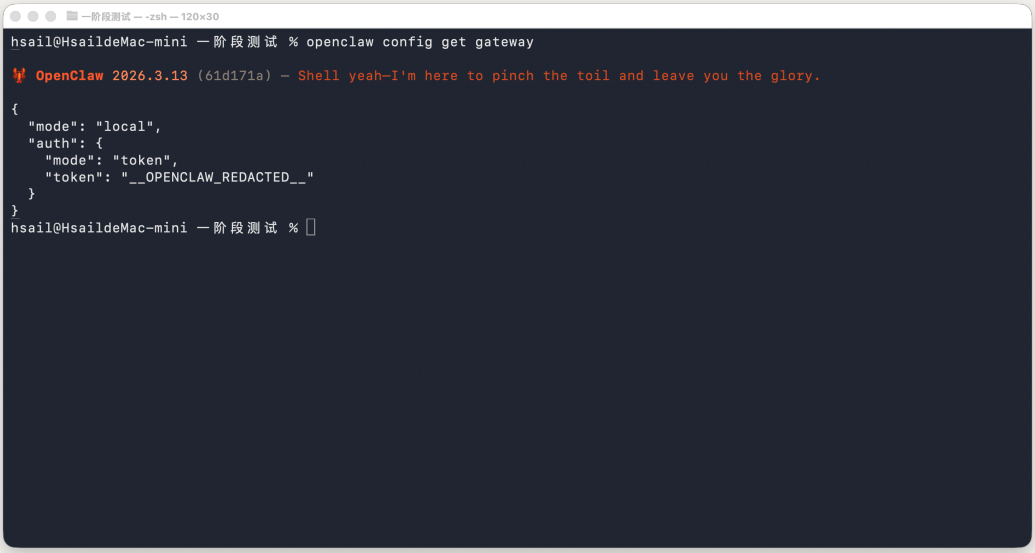
模式	广播内容	安全性
`full`	角色、端口、CLI 路径、SSH 端口	低（泄露路径信息）
`minimal`	角色、端口	中（默认）
`off`	不广播	高

如果你只在自己电脑上使用，可以设为 `off` 彻底关闭广播。

实操：确认当前网络安全配置

来看看你的 Gateway 当前的网络配置：

```
$ openclaw config get gateway
```

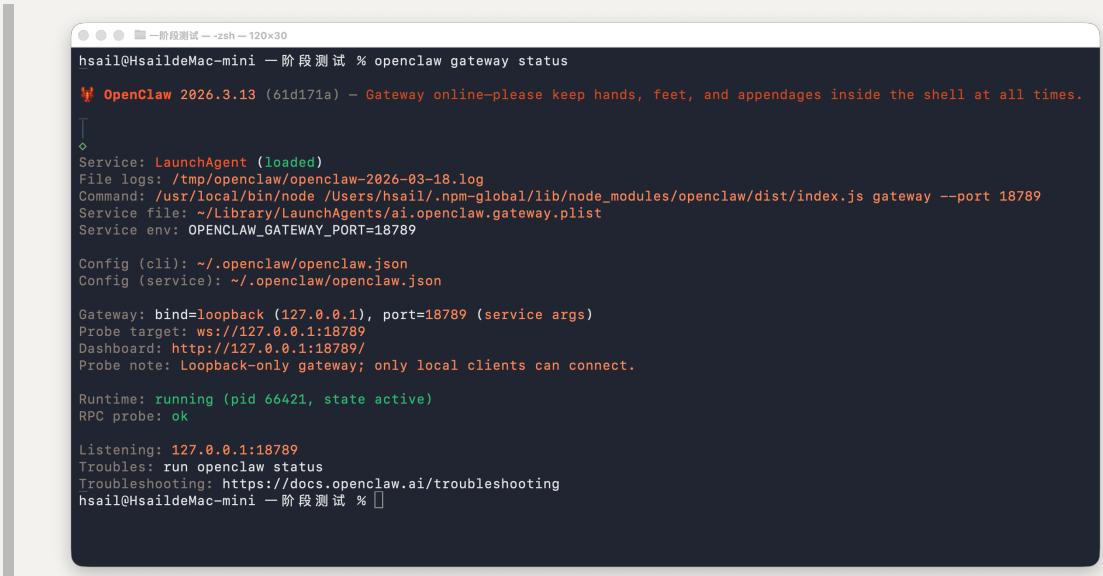


关键信息解读：

字段	当前值	评价
`mode`	`local`	本地模式
`auth.mode`	`token`	Token 认证
`auth.token`	`a261b163...`	已自动生成

再来看 Gateway 的运行状态：

```
$ openclaw gateway status
```



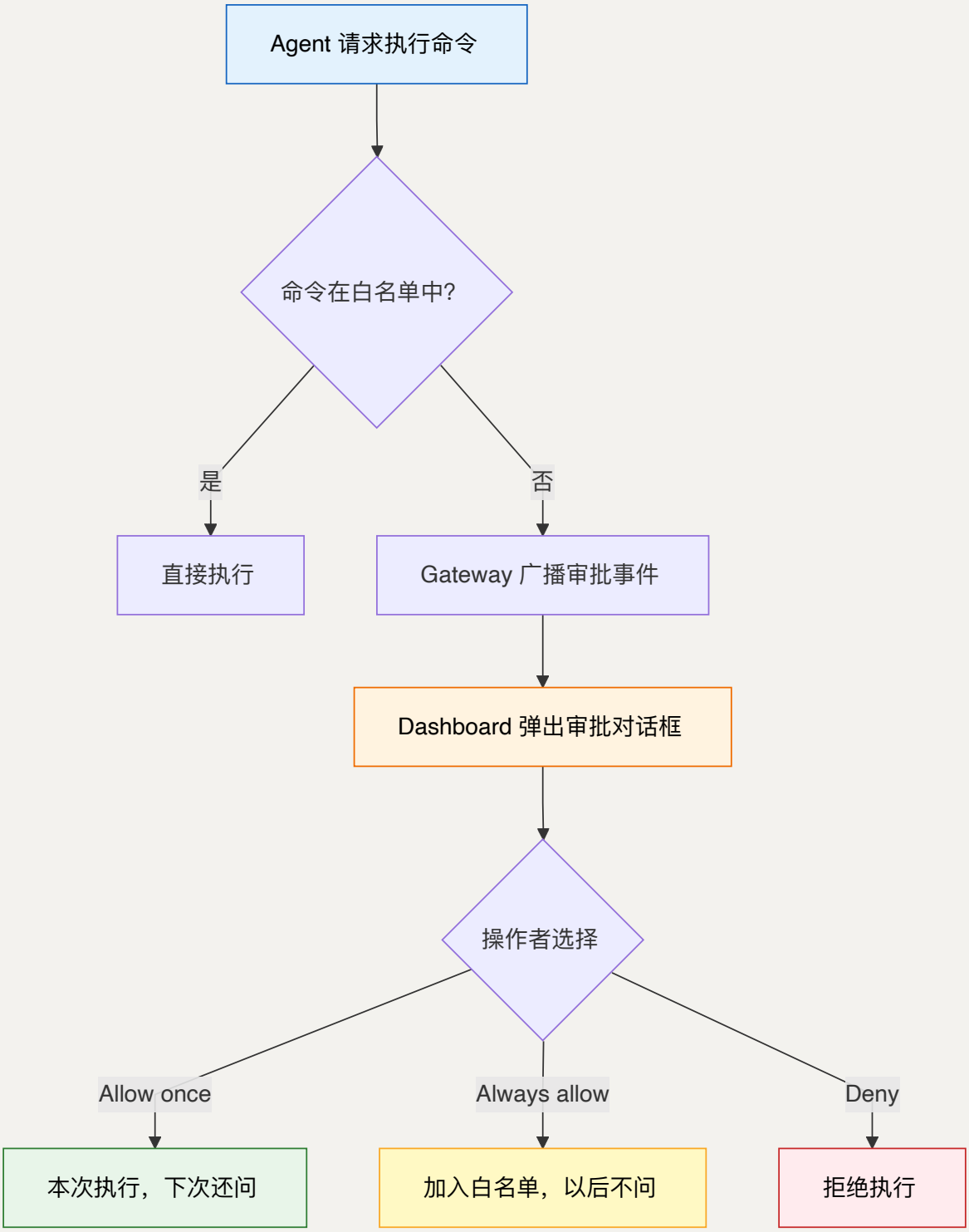
看到 `bind=loopback (127.0.0.1)` 和 `Probe note: Loopback-only gateway; only local clients can connect.`——你的 **Gateway** 绑定配置是安全的。只有你自己的电脑能连上来，外部网络无法访问。

3.5 行为控制：Exec Approval 执行审批机制

权限控制决定"AI 能不能做"，而 Exec Approval 决定"AI 做之前要不要问你"。这是 OpenClaw 最核心的行为控制机制。

审批流程

当 AI 想执行一个命令时，会经历这样的流程：



审批对话框会展示完整的信息：

- **命令内容**：AI 想执行什么命令
- **参数**：命令的具体参数
- **工作目录**：在哪个路径下执行
- **Agent ID**：哪个 Agent 发起的请求

你有三个选择：

选择	效果	适用场景
Allow once	允许这一次，下次执行同样的命令还会再问	不确定是否安全时先试一次
Always allow	加入白名单，以后这个命令直接放行	确认安全的常用命令
Deny	拒绝执行，AI 会收到拒绝通知	明显不安全或不需要的操作

关闭全自动执行的重要性

OpenClaw 配置中有一个选项 `tools.exec.ask: "off"`，作用是**永远不弹审批框、所有命令静默执行**。

这在个人电脑上是极度危险的配置。

想象一下：你让 AI "帮我整理一下桌面的文件"，它可能执行 `rm -rf ~/Desktop/*`——如果没有审批机制，你的桌面文件就直接没了，没有任何确认、没有回退的机会。

更危险的场景：如果有人通过社会工程（比如伪装成正常对话）让你的 AI 执行了恶意命令——没有审批就没有最后的防线。

> *底线：除非你在一个完全隔离的测试环境中，否则不要把 `ask` 设为 `off`。推荐 `on-miss`（日常使用）或 `always`（高安全要求）。*

3.6 成本控制：防止账单失控

安全不只是防黑客——防止自己的钱包被掏空也是安全的一部分。

OpenClaw 使用外部大模型 API，按 Token 计费。如果不加控制，一个复杂任务可能在你不知情的情况下消耗大量 Token，产生意想不到的账单。

成本控制核心配置

OpenClaw 通过多个配置项组合控制成本：

配置项	作用	推荐值	说明
`maxTokens`	限制单次输出 Token 数	8192	防止 AI 生成过长的回复
`contextTokens`	限制上下文窗口大小	50000-100000	减少输入 Token 消耗
`maxConcurrent`	限制并发 Agent 数	3-4	防止同时运行太多 Agent
`timeoutSeconds`	Agent 超时时间	600	10 分钟没响应就停止

当前环境的配置状态：

```
{
  "agents": {
    "defaults": {
      "model": { "primary": "deepseek/deepseek-chat" },  // DeepSeek Chat
      // maxTokens: 8192 (在 provider 中配置)
      // contextWindow: 128000
    }
  }
}
```

Thinking 模式：Token 消耗的"隐形炸弹"

有一个特别容易踩的坑——**推理模式（Thinking Mode）**。

一些模型（如 DeepSeek Reasoner、Gemini 2.5 Flash）支持"推理模式"，让 AI 在回答前先"思考"一番。思考过程会产生大量的推理 Token。

关键数字：**推理模式的 Token 消耗是普通模式的 10-50 倍**。

也就是说，一个本来消耗 1000 Token 的问题，开了推理模式可能变成 50000 Token。按 DeepSeek 的价格算，输出成本从约 0.001 美元变成 0.05 美元——看似不多，但如果你一天问 100 个问题，差距就是 $0.10 vs $5.00。

如果你用的是价格更高的模型（如 GPT-5.4），这个差距会更加夸张。

关闭推理模式的配置：

```
{
  "agents": {
    "defaults": {
      "models": {
        "deepseek/deepseek-reasoner": {
          "params": {
            "thinking": { "type": "disabled" }    // 关闭推理模
式
          }
        }
      }
    }
  }
}
```

费用优化策略速查表

策略	配置方式	预期效果
缩小上下文	`contextTokens: 50000`	减少输入 Token
限制输出	`maxTokens: 8192`	减少输出 Token
关闭推理	`thinking.type: "disabled"`	减少 10-50 倍 Token
降低图片分辨率	`imageMaxDimensionPx: 800`	减少视觉 Token
选择低价模型	如 Flash 替代 Pro	降低单价
限制并发	`maxConcurrent: 1`	控制总量
设置超时	`timeoutSeconds: 600`	防止卡死消耗

> *个人建议：对于日常使用，DeepSeek Chat 的性价比极高（500 万免费 Token + 极低单价）。如果你发现费用增长异常，首先检查是不是不小心开了推理模式。*

3.7 安全基线配置总结

学了这么多维度的安全配置，最后来一份"一键套用"的安全基线——这是我们综合前面所有知识给出的**推荐配置**，适合个人开发环境使用。

推荐安全基线（课程附赠）

```
{
  // 🌐 网络：仅本机访问 + Token 认证
  "gateway": {
    "mode": "local",
    "bind": "loopback",
    "port": 18789,
    "auth": {
      "mode": "token"
      // token 由 openclaw gateway install 自动生成
    }
  },

  // 🔑 会话：每个用户+渠道独立隔离
  "session": {
    "dmScope": "per-channel-peer"
  },

  // 🛠️ 工具：编程 Profile + 不在白名单时确认
  "tools": {
    "profile": "coding",
    "exec": {
      "ask": "on-miss"                // 已知命令放行，新命令弹确认
      // security 不额外限制（coding profile 已有合理默认）
    }
  },

  // 📡 发现：最小广播
  "discovery": {
    "mdns": { "mode": "minimal" }
  },

  // 🤖 Agent：合理的资源限制
  "agents": {
    "defaults": {
      "contextTokens": 100000,       // 上下文窗口限制
      "maxConcurrent": 3,            // 最多 3 个并发 Agent
      "timeoutSeconds": 600          // 10 分钟超时
    }
  }
```

```
}
```

这份配置的安全策略可以用一句话概括：**本机访问 + Token 认证 + 命令审批 + 资源限制**。

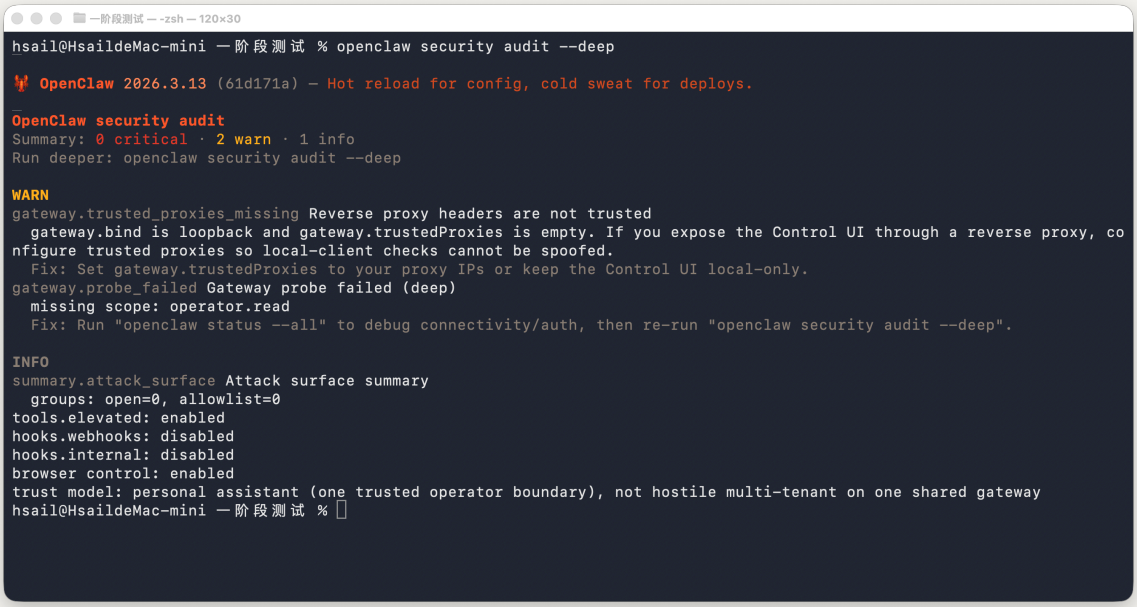
如果你需要更高的安全级别（比如部署在服务器上给团队使用），可以在此基础上加强：

- `tools.exec.ask` 改为 `"always"`
- `discovery.mdns.mode` 改为 `"off"`
- 启用 Docker 沙箱：`agents.defaults.sandbox.mode: "all"`
- 缩小工具白名单：`tools.deny` 添加更多工具

实操：运行安全审计

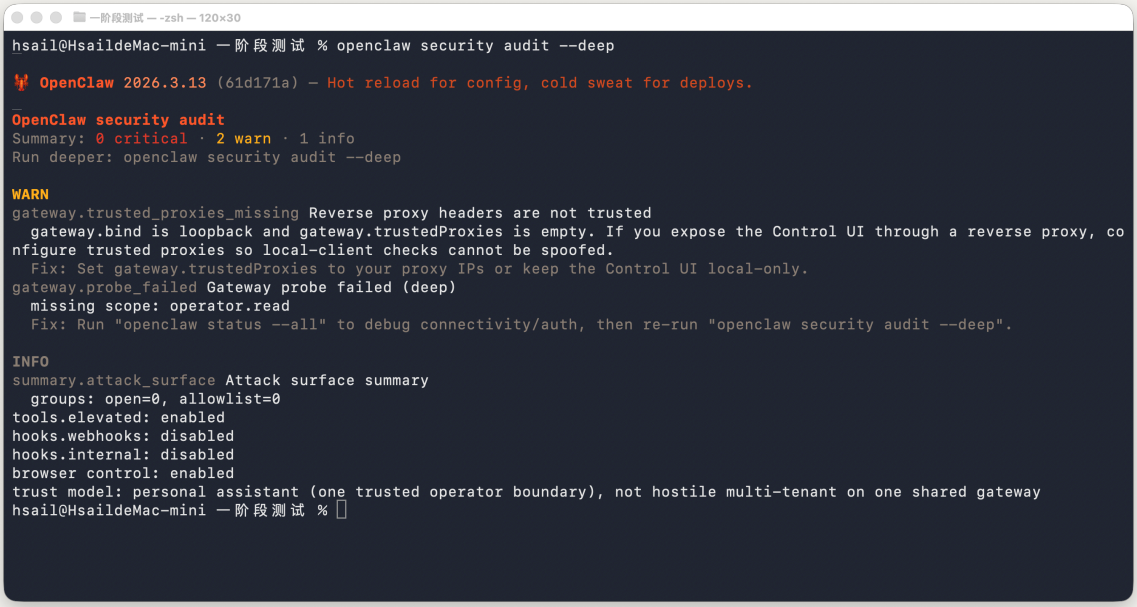
OpenClaw 自带安全审计工具，可以帮你检查当前配置是否存在安全隐患：

```
$ openclaw security audit --deep
```



审计工具会连接 Gateway 进行探测：

审计完成后，输出结果：



来解读审计结果：

```
Summary: 0 critical · 2 warn · 1 info
```

级别	项目	说明	是否需要修复
WARN	trusted_proxies_missing	未配置可信代理 IP	如果不用反向代理，可忽略
WARN	probe_failed	深度探测权限不足	通常是 token scope 问题，可忽略
INFO	attack_surface	攻击面摘要	仅供参考

0 critical（零严重问题）——这是最重要的数字。说明你的 OpenClaw 当前没有严重的安全风险。

两个 warn 都是"如果你做了某些操作才需要关注"的场景：

- `trusted_proxies_missing`：只有当你通过 Nginx 等反向代理暴露 Gateway 时才需要配置
- `probe_failed`：深度探测需要特定的 token scope，个人使用场景不影响

INFO 中的攻击面摘要显示：

- `groups: open=0`：没有开放的群组入口
- `tools.elevated: enabled`：提权功能开启，但被 gate 拦截

- `trust model: personal assistant`：个人助手信任模型

总体评价：**当前配置符合个人开发环境的安全要求**。

安全检查习惯

建议养成定期运行安全审计的习惯——特别是在以下场景之后：

- 安装了新的 Skill
- 修改了 Gateway 配置
- 更新了 OpenClaw 版本
- 开放了新的消息通道

一条命令，几秒钟时间，换来的是安心。